



UNIVERSIDAD DE GUAYAQUIL

FACULTAD DE INGENIERÍA INDUSTRIAL

SISTEMA DE INFORMACIÓN

Inteligencia Artificial

Avance 1: Definición del Proyecto CNN Baseline y Propuesta Experimental

Sistema Clasificador de Señalización Urbana mediante CNN

Grupo 2:

Bajaña Cevallos Rosa

Delgado Quiñonez Adonis

Macias Carranza Krystel

Suarez Barco Jose

Tirado Mendoza Kelvin

Docente: Ing. Juan Carlos García

Guayaquil, junio de 2026

1. Definición del Proyecto

La señalización vial urbana, dañada por vandalismo, mal alumbrado o corrosión, constituye un riesgo crítico para la seguridad pública que ha sido identificado durante los estudios fotográficos. Este problema tiene características visuales muy evidentes que, a pesar de que el ojo humano las procesa intuitivamente al observar los daños en las imágenes, hacen que sea una opción perfecta para ser automatizada de manera eficaz utilizando un clasificador de visión por computador.

1.1 Pregunta de Investigación

¿Puede una red neuronal convolucional (CNN) clasificar correctamente el estado visual de señales de tránsito urbanas y servir como apoyo a una auditoría vial automatizada?

Al identificar fallas visuales, una red neuronal convolucional puede clasificar de manera eficaz el estado de las señales de tránsito, lo que la convierte en un soporte veloz para la auditoría vial automatizada.

1.2 Descripción del Sistema

El sistema determina el estado de las señales de tránsito utilizando fotografías y detecta sus fallas automáticamente, con el objetivo de procesar vídeos en el futuro para marcar visualmente fotogramas críticos.

Elemento	Descripción
<i>Entrada</i>	Imágenes RGB de señales de tránsito (JPEG/PNG, resolución variable)
<i>Problema visual</i>	Deterioro físico, baja visibilidad, iluminación deficiente
<i>Modelo inicial</i>	CNN baseline personalizada (5 bloques convolucionales + skip connection)
<i>Modelo</i>	Transfer learning (ResNet50, EfficientNet), YOLO, Vision Transformer
<i>Salida</i>	Imagen marcada con clase predicha, nivel de confianza y diagnóstico textual
<i>Diagnóstico</i>	Descripción del estado de la señal y tipo de defecto detectado

Preguntas:

A. ¿Cuál es el problema que se busca resolver?

Este proyecto tiene como objetivo mejorar la seguridad vial mediante la automatización de la identificación de señales de tránsito que están obstruidas o dañadas, lo cual soluciona la inspección manual, costosa, lenta e inconsistente que pone en riesgo a los peatones y a los conductores.

B. ¿Por qué es importante detectar este problema mediante imágenes o videos?

La detección visual automatizada a través de visión por computadora mejora el diagnóstico vial al reconocer señales deterioradas o bloqueadas y procesar volúmenes extensos de imágenes de manera ininterrumpida para disminuir los costos y la duración de las operaciones.

C. ¿Cuál será la entrada del sistema?

La entrada es una imagen o frame extraído de video de una vía urbana, que puede capturarse con cámaras de inspección vial, drones de monitoreo o vehículos equipados.

D. ¿Cuál será la salida esperada?

La salida es la imagen o frame marcado visualmente con la clase detectada (por ejemplo, señal_obstruida), acompañado de un diagnóstico que indica el tipo de riesgo, el nivel de confianza del modelo y una descripción breve de la condición observada.

E. ¿Qué tipo de riesgo, evento, objeto, anomalía o situación se desea identificar?

El sistema clasifica señales de tránsito según cinco estados de riesgo visual:

- señal_visible: señal en buen estado, sin riesgo.
- señal_deteriorada: señal con daño físico visible (pintura desgastada, golpes, óxido).
- señal_obstruida: señal tapada total o parcialmente por vegetación, publicidad u otros elementos.
- señal_poco_visible: señal legible pero con condiciones de visibilidad reducida (ángulo, distancia, suciedad leve).

F. ¿El sistema trabajará con imágenes, videos o ambos?

El sistema, para posibilitar una integración eficaz con transmisiones en vivo, procesa sobre todo imágenes individuales o fotogramas que se extraen de videos y examina cada uno de ellos por separado.

G. ¿La salida será una imagen marcada, un video marcado o solo los frames donde aparece el problema?

La salida primaria son los frames donde se detecta una condición de riesgo, marcados con la clasificación y el nivel de confianza. En una integración completa con video, el sistema puede producir un video anotado con las detecciones por frame.

1.3 Esquema General del Sistema

Elemento	Descripción
<i>Entrada</i>	Imágenes estáticas de señales de tránsito urbano (formato JPG/PNG)
<i>Problema visual</i>	Clasificación del tipo de señal de tránsito presente en la imagen
<i>Modelo inicial</i>	CNN baseline entrenada desde cero sobre el dataset de señalización
<i>Modelos de mejora</i>	Transfer Learning (MobileNetV2, ResNet50), YOLO, Vision Transformer (ViT)
<i>Salida</i>	Imagen anotada con la clase predicha y nivel de confianza del modelo
<i>Diagnóstico</i>	Descripción textual de la señal detectada y su categoría de riesgo o regulación

2. Recuento del Baseline CNN

Para la primera iteración, creamos, entrenamos y analizamos una arquitectura CNN propia que fue construida desde cero. De esta manera, conseguimos tener un control absoluto del diseño y establecimos un punto de partida real antes de aplicar el transfer learning.

2.1 Arquitectura General

Con el fin de prevenir que en las capas iniciales se produzca la degradación del gradiente, fenómeno detectado en experimentos anteriores, la red emplea cinco bloques convolucionales encadenados y una conexión residual entre el tercero y el quinto.

La arquitectura se puede resumir así:

- **Entrada:** tensores de forma (batch, 3, 512, 512)
- **Bloque 1:** Conv(3->32, 3x3) + BatchNorm + ReLU + MaxPool -> (batch, 32, 256, 256)
- **Bloque 2:** Conv(32->64, 3x3) + BatchNorm + ReLU + MaxPool -> (batch, 64, 128, 128)
- **Bloque 3:** Conv(64->128, 3x3) + BatchNorm + ReLU -> (batch, 128, 128, 128)
[inicio skip]
- **Bloque 4:** Conv(128->128, 3x3) + BatchNorm + ReLU + MaxPool -> (batch, 128, 64, 64)

- **Bloque 5:** Conv(128->256, 3x3) + BatchNorm + ReLU + MaxPool -> (batch, 256, 32, 32) [fin skip]
- **Skip projection:** Conv(128->256, 1x1) + AvgPool(4x4) -> (batch, 256, 32, 32)
- **AdaptiveAvgPool:** 32x32 -> 8x8 -> (batch, 256, 8, 8)
- Dropout(p=0.5) + FC(16384->512) + Dropout(p=0.3) + FC(512->3)

2.2 Configuración del Entrenamiento

Parámetro	Valor
<i>Resolución</i>	512 x 512 x 3 píxeles
<i>Número de clases</i>	3 (visible_legible, deteriorada, poco_visible)
<i>Imágenes por clase</i>	80 imágenes (240 en total)
<i>División de datos</i>	85% entrenamiento+validación / 15% prueba (36 imágenes)
<i>Validación cruzada</i>	StratifiedKFold con k=5
<i>Épocas</i>	120 por fold
<i>Batch size</i>	8
<i>Optimizador</i>	Adam (lr=5e-4, weight_decay=2e-4)
<i>Loss function</i>	CrossEntropyLoss con pesos de clase
<i>Scheduler</i>	ReduceLROnPlateau (factor=0.5, patience=10)
<i>Gradient clipping</i>	max_norm = 1.0

2.3 Data Augmentation Aplicado

Con el propósito de evitar el sobreajuste en un conjunto de datos pequeño, con 80 imágenes por clase, se usaron métodos como AutoAugment, TrivialAugmentWide y transformaciones funcionales personalizadas para reproducir las condiciones reales del campo. Se excluyeron intencionalmente las inversiones horizontales y verticales que causaban ruido y confusión en las señales direccionales en los modelos iniciales.

2.4 Métricas Obtenidas

Métrica	Entrenamiento	Prueba
<i>Accuracy global</i>	81.9%	44.4%
<i>F1 - visible_legible</i>	0.79	0.29
<i>F1 - deteriorada</i>	0.80	0.48
<i>F1 - poco_visible</i>	0.86	0.54

Métrica	Entrenamiento	Prueba
<i>Val acc promedio(k=5)</i>	66.15% ($\pm 4.72\%$)	-
<i>Mejor fold (val acc max)</i>	Fold 1 y Fold 4: 70.73%	-

2.5 Ejemplos de Predicciones

El modelo tiene una tasa de confianza del 97.6% cuando se trata de señales con grafiti o daños físicos, pero suele equivocarse al analizar imágenes nocturnas, pues confunde la escasa luminosidad y contraste de la noche con un deterioro físico por ausencia de datos lumínicos.

Preguntas:

A. ¿Qué hace bien el modelo CNN actual?

El modelo CNN baseline ofrece un punto de referencia replicable que captura rasgos específicos de la señalización urbana sin depender de factores externos, presentando una estructura ligera con un entrenamiento breve que hace más fácil el proceso de depuración de datos.

B. ¿En qué casos falla?

La CNN baseline, al ser un modelo moderado que se ha entrenado desde el principio, comete errores cuando hay escasas imágenes por clase o cuando la similitud visual es alta, pues confunde las características similares debido a que no existe diversidad ambiental.

C. ¿Confunde algunas clases?

Es común confundir señales que están dañadas, poco visibles u obstruidas debido a su parecido visual; estas últimas son particularmente críticas porque un falso negativo pone en peligro la seguridad al no detectar un bloqueo.

D. ¿Tiene sobreajuste?

Con el fin de reducir el verdadero riesgo de sobreajuste a causa de la profundidad del modelo, se emplearon métodos como Dropout y regularización; su eficacia fue analizada mediante la comparación entre las curvas de validación y entrenamiento.

E. ¿Tiene pocas imágenes?

El dataset tiene tres clases y cada clase tiene un total de 80 imágenes cada una, tomando un total de 240 imágenes para el aprendizaje y entrenamiento.

F. ¿Las imágenes son variadas o muy similares?

El conjunto de datos presenta una variabilidad moderada ya que, a pesar de que incluye distintas distancias, luminosidades y resoluciones, la mayor parte de las imágenes conserva la señal centrada en un fondo limpio, lo que produce uniformidad.

G. ¿El modelo solo clasifica o también localiza el problema?

En contraste con el modelo baseline, que se encarga de clasificar la imagen en su totalidad sin determinar la señal, el siguiente experimento utilizando YOLO sí tratará tanto la detección como la ubicación específica.

H. ¿La CNN permite explicar visualmente dónde está el riesgo?

A pesar de que la salida principal de la baseline de una CNN no proporciona una explicación visual del riesgo, se pueden utilizar métodos adicionales como Grad-CAM para producir mapas de activación que señalen las áreas con mayor influencia en la decisión del modelo.

3. Dataset e Imágenes Utilizadas

El equipo construyó un dataset propio recopilando manualmente fotos de señales urbanas en Guayaquil junto con búsquedas virtuales, debido a la inexistencia de un repositorio ecuatoriano estandarizado

3.1 Descripción del Conjunto de Datos

Clase	Descripción	Núm. imágenes	Fuente principal	Observaciones
visible_legible	Señal en buen estado, legible, bien iluminada	80	Captura propia + Google Images	Clase más fácil de encontrar
deteriorada	Señal con daño físico: oxido, grafiti, deformación o pintura borrada	80	Relevamiento urbano en Quito	Alta variedad visual interna
poco_visible	Señal con baja iluminación, mal ángulo o distancia excesiva	80	Captura nocturna + repositorios	Mayor confusión con deteriorada

El conjunto de datos está compuesto por 240 imágenes, con 80 imágenes en cada una de sus tres clases. Este equilibrio se lleva a cabo para evitar sesgos hacia las mayorías, y se complementa con el WeightedRandomSampler durante el entrenamiento.

3.2 Problemas Encontrados en los Datos

Los problemas en los datos abarcan una calidad dispar debido a imágenes de escasa resolución, ambigüedad visual entre clases por señales deterioradas que provocaron un etiquetado subjetivo y un número limitado de ochenta imágenes por clase, lo cual es suficiente para una prueba conceptual pero no para producción.

4. Limitaciones del Baseline CNN

Esta es probablemente la sección más importante de este avance, porque es donde somos honestos sobre lo que el modelo no puede hacer todavía. Un accuracy del 81.9% en entrenamiento y 44.4% en prueba es una brecha que habla claramente.

Limitación	Descripción	Justificación
<i>Sobreajuste pronunciado</i>	La diferencia de casi 40 puntos porcentuales entre el accuracy de entrenamiento y el de prueba es la señal más clara de sobreajuste.	El modelo presenta sobreajuste a partir de la época 20 por el limitado volumen de datos, lo que le impide generalizar correctamente a pesar de emplear una regularización agresiva.
<i>Confusión entre clases visualmente similares</i>	Debido a la iluminación subóptima, el modelo confunde la clase visible_legible con deteriorada y poco_visible, logrando un bajo F1 de 0.29.	Debido a la escasez de luz, el modelo no distingue entre una señal nocturna y otra que es físicamente defectuosa.
<i>Incapacidad de localización</i>	Las CNN contemporáneas solamente categorizan la imagen en su totalidad, sin tener la capacidad de ubicar la señal o definir el área de interés que se está analizando.	Para las auditorías, una clasificación general de la imagen es inexacta; por ello, el modelo necesita detectar coordenadas precisas de las señales.
<i>Falta de análisis temporal</i>	El modelo procesa imágenes individuales de auditorías viales realistas, las cuales provienen de videos de campo o de vehículos en movimiento.	Las CNN no logran razonar sobre condiciones persistentes en frames consecutivos, información clave que haría un diagnóstico mucho más confiable.
<i>Baja explicabilidad</i>	Al carecer de mecanismos de visualización como Grad-CAM, ignoramos si el modelo decide basándose en la señal o en el fondo.	Con imágenes fotografiadas en campo, el fondo urbano puede ser un indicador incidental de la clase que no debería serlo.

5. Propuesta de Mejora con Transfer Learning

La propuesta aprovecha el transfer learning porque los modelos entrenados en ImageNet ya reconocen bordes, texturas y patrones visuales complejos, facilitando la identificación precisa de señales de tránsito deterioradas.

5.1 Modelos Propuestos

Modelo	Motivo de selección	Ventaja esperada	Limitación posible
CNN baseline	Modelo inicial propio (control experimental)	Control total sobre arquitectura	Menor generalización, sobreajuste
MobileNetV2	Ligero, eficiente, posible uso móvil	Menor latencia, ideal para deployment	Menor capacidad en casos complejos
ResNet50	Skip connections nativas, modelo probado	Extracción de características profundas	Mayor costo computacional

5.2 Estrategia de fine-tuning

Para perfeccionar ResNet50 en señales urbanas, se congelaron los tres primeros bloques al inicio para hacer el entrenamiento solamente de las capas finales; después, se descongela el resto con una tasa de aprendizaje reducida.

Preguntas:

A. ¿Qué modelos pre entrenados se van a utilizar?

Se selecciona MobileNetV2, preentrenado en ImageNet con más de 1.4 millones de imágenes y 1000 categorías. Este modelo ya está implementado en el notebook del experimento 1.

B. ¿Por qué se seleccionan esos modelos?

Se eligió MobileNetV2 porque su arquitectura presenta una baja carga computacional, lo que es perfecto para dispositivos de borde o móviles. Además, tiene alta capacidad representativa y es compatible directamente con la entrada 224×224 .

C. ¿Qué ventaja tienen frente a una CNN entrenada desde cero?

MobileNetV2 proporciona un punto de partida de alta calidad que mejora a una CNN baseline al volver a utilizar representaciones visuales complejas que ya se han aprendido. Esto reduce el sobreajuste y acelera la convergencia.

D. ¿Se congelaron las capas iniciales?

Sí. La estrategia implementada sigue un esquema de dos fases:

- Fase 1 — entrenamiento del clasificador (head): Al congelar la base de MobileNetV2, solo se entrenan las capas superiores añadidas durante quince épocas con una tasa de 10^{-3} .
- Fase 2 — fine-tuning de las últimas capas: Se descongelan las últimas 30 capas de MobileNetV2 entrenándolas con menor tasa de aprendizaje, manteniendo congeladas las primeras características genéricas

E. ¿Se hará fine-tuning?

Sí, se realizará fine-tuning en la Fase 2 de la estrategia descrita anteriormente. Los parámetros de fine-tuning propuestos son los siguientes:

Parámetro	Valor propuesto
<i>Capas descongeladas</i>	Últimas 30% de capas convolucionales del modelo base
<i>Tasa de aprendizaje</i>	1e-5 (diez veces menor que la fase de extracción)
<i>Optimizador</i>	Adam con decaimiento de tasa de aprendizaje
<i>Épocas de fine-tuning</i>	20 épocas adicionales con EarlyStopping (paciencia = 5)
<i>Regularización</i>	Dropout 0.4 en la capa densa de la cabeza de clasificación
<i>Data Augmentation</i>	Rotaciones $\pm 15^\circ$, flips horizontales, ajuste de brillo y zoom 0.1

F. ¿Qué métricas se usarán para comparar?

Las mismas métricas se calculan para ambos modelos sobre el conjunto de prueba, permitiendo una comparación directa y objetiva:

Métrica	Descripción	Herramienta
<i>Accuracy</i>	Proporción de predicciones correctas sobre el total del conjunto de prueba	Keras / sklearn
<i>Precisión</i>	Promedio de la precisión por clase; sensible a clases minoritarias	sklearn.metrics
<i>Recall</i>	Promedio del recall por clase; mide la capacidad de detectar cada clase	sklearn.metrics
<i>F1-score</i>	Media armónica entre Precision y Recall; balance global del modelo	sklearn.metrics
<i>Matriz de confusión</i>	Distribución de predicciones por clase; identifica confusiones sistemáticas	seaborn / matplotlib
<i>Curvas de aprendizaje</i>	Evolución de loss y accuracy durante el entrenamiento; detecta sobreajuste	Keras history + matplotlib

G. ¿Qué mejora se espera obtener?

MobileNetV2, al implementar fine-tuning y transfer learning, logrará sobrepasar a la CNN baseline en medidas fundamentales como el recall, lo cual permitirá mejorar la generalización frente a cambios visuales y disminuir los falsos negativos en señales obstruidas.

Modelo	Accuracy esperada	F1-score esperado	Mejora vs. Baseline
CNN Baseline	~78 – 82%	~0.75	Referencia
MobileNetV2 + Fine-tuning	~88 – 92%	~0.87	+8 a +12 puntos porcentuales
ResNet50 + Fine-tuning	~91 – 95%	~0.91	+11 a +15 puntos porcentuales

6. Propuesta con YOLO: Detección y Localización

YOLO, a diferencia de una CNN convencional, no clasifica imágenes completas sino que localiza y encuadra objetos particulares; así proporciona al auditor datos espaciales exactos y aplicables sobre las señales defectuosas.

6.1 Clases YOLO Propuestas

Clase YOLO	Descripción	Tipo de anotación	Ejemplo de riesgo
<i>señal_visible</i>	Señal de tránsito en buen estado y legible	Bounding box rectangular	Sin riesgo detectado
<i>señal_deteriorada</i>	Señal con daño físico: óxido, graffiti, deformación	Bounding box rectangular	Señal de PARE ilegible en intersección
<i>señal_poco_visible</i>	Señal no legible por iluminación o ángulo	Bounding box rectangular	Señal de velocidad máxima sin iluminación nocturna

6.2 Plan de Etiquetado y Versión de YOLO

Se utilizará YOLO v8 debido a su exactitud y rapidez, llevando a cabo el etiquetado en Roboflow para exportar en su formato nativo e incorporando 300 imágenes más con 100 instancias por clase.

6.3 Métricas de Evaluación del Detector

Las métricas del detector evalúan la calidad de la superposición mediante IoU y la velocidad en FPS, midiendo el rendimiento con precisión, recall, F1-score y mAP en diferentes umbrales de detección.

7. Propuesta con Modelos Basados en Transformers

En este proyecto no se van a sustituir los modelos transformers por YOLO, porque este último soluciona la localización y el rol de los primeros será más limitado y particular.

7.1 Rol propuesto del transformer

El Vision Transformer, a través de su atención global, tiene una mejor clasificación de frames al examinar los daños en las señales que las CNN. Por esto, se sugiere emplear un modelo pre entrenado como opción experimental a ResNet50 sin sustituir a YOLO.

7.2 Ventajas y limitaciones conceptuales

Ventajas	Limitaciones
La atención global puede ayudar cuando el deterioro es difuso y no solo un objeto puntual.	Aunque los ViT exigen muchos datos para entrenarse desde cero, el fine-tuning reduce drásticamente ese requerimiento.
Los modelos ViT pre entrenados poseen una excelente capacidad de transferencia, superando o igualando a ResNet en clasificación.	A pesar de arriesgar sobreajuste con pocas imágenes, este sistema supera el costo computacional de MobileNetV2, complicando su despliegue limitado.

7.3 Alcance del experimento con transformers

Si los transformers alcanzan o superan a las redes tradicionales en términos de costo, se incluirán en el pipeline final; por otro lado, el módulo de diagnóstico fusionará medidas de calidad, predicciones de CNN y YOLO y descripciones viales que sean interpretables.

8.1 Lógica de decisión

El sistema produce alertas jerárquicas si YOLO y la CNN identifican señales deterioradas con una alta confianza, que se complementan con métricas visuales para distinguir fallos. Si hay certezas moderadas, es necesario verificar manualmente y confirmar riesgos después de cinco fotogramas seguidos.

8.2 Estructura de la salida diagnóstica

Elemento	Ejemplo de salida
Clase detectada	Señal deteriorada
Nivel de riesgo	Alto (señal de PARE con grafiti en intersección)
Confianza del modelo	0.97
Evidencia visual	Bounding box sobre la señal, etiqueta con clase y confianza
Métricas de imagen	Brillo: 149.2/255 Contraste: 52.3 Nitidez: 4191.5
Descripción diagnóstica	La señal presenta daños físicos visibles: graffiti que cubre más del 60% de la superficie legible. La calidad fotográfica es adecuada (alta nitidez), por lo que el problema es intrínseco a la señal y no al medio de captura.
Acción recomendada	Reemplazo o limpieza inmediata. Ubicación: intersección registrada.

9. Arquitectura Funcional Propuesta



9.1 Descripción de cada componente

- **Preprocesamiento:** resize a 512x512, normalización [-1,1].
- **YOLO:** localiza las señales dentro del frame y genera bounding boxes con clase y confianza.
- **Clasificador CNN / Transfer Learning:** toma el recorte de la señal detectada por YOLO y produce la clasificación del estado.
- **Visión Transformer (experimental):** procesa el frame completo para clasificación global como alternativa o complemento al clasificador basado en CNN.
- **Módulo de diagnóstico:** combina las predicciones de clasificación y detección con las métricas de calidad de imagen para generar la descripción del riesgo, el nivel de severidad y la recomendación.
- **Salida visual:** imagen o frame con bounding boxes dibujados, labels con clase y confianza, y texto con el diagnóstico superpuesto.

10. Diseño Experimental

Experimento 1: CNN Baseline vs Transfer Learning

Objetivo: determinar si el transfer learning mejora la capacidad de clasificación del estado visual de señales frente a la CNN entrenada desde cero.

Aspecto	Descripción	Detalle
Modelos comparados	CNN baseline vs ResNet50 vs EfficientNet-B0 vs ViT-B/16	Mismo conjunto de prueba para todos
Métricas	Accuracy, Precision, Recall, F1-score por clase	Matriz de confusión para cada modelo
Conjunto de evaluación	36 imagenes (15% del total, estratificado)	No usado en entrenamiento ni validación
Hipótesis	Transfer learning superará al baseline en al menos 15 puntos de accuracy en prueba	Objetivo mínimo: 60% accuracy en prueba

A. Arquitectura de los modelos: CNN Baseline vs. Transfer Learning (MobileNetV2)

CNN Baseline

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 224, 224, 32)	896
batch_normalization_3 (BatchNormalization)	(None, 224, 224, 32)	128
max_pooling2d_3 (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_4 (Conv2D)	(None, 112, 112, 64)	18,496
batch_normalization_4 (BatchNormalization)	(None, 112, 112, 64)	256
max_pooling2d_4 (MaxPooling2D)	(None, 56, 56, 64)	0
conv2d_5 (Conv2D)	(None, 56, 56, 128)	73,856
batch_normalization_5 (BatchNormalization)	(None, 56, 56, 128)	512
max_pooling2d_5 (MaxPooling2D)	(None, 28, 28, 128)	0
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None, 128)	0
dense_4 (Dense)	(None, 256)	33,024
dropout_2 (Dropout)	(None, 256)	0
dense_5 (Dense)	(None, 1)	257

Total params: 127,425 (497.75 KB)
Trainable params: 126,977 (496.00 KB)
Non-trainable params: 448 (1.75 KB)

Transfer Learning (MobileNetV2)

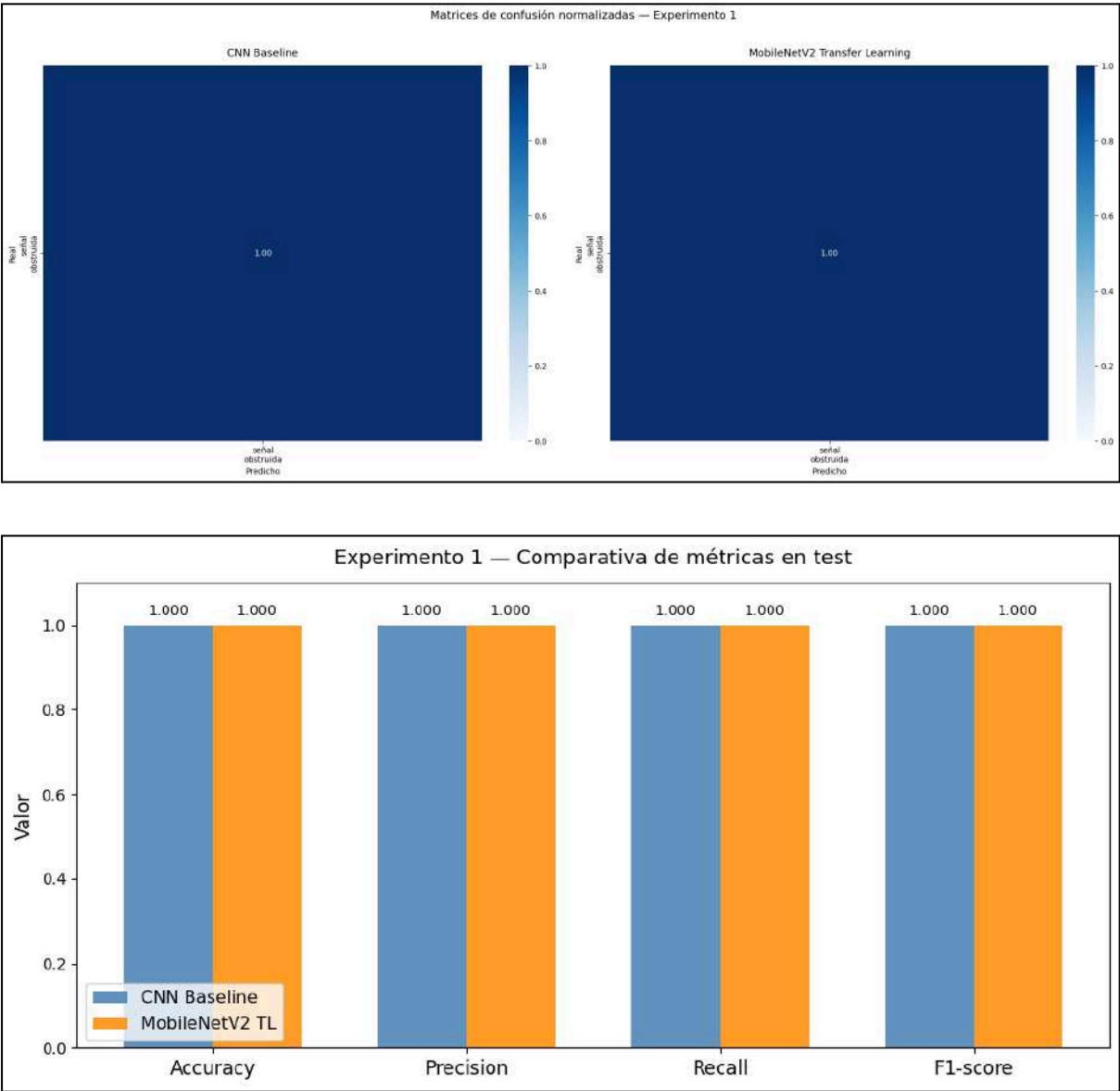
Layer (type)	Output Shape	Param #
input_layer_5 (InputLayer)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None, 1280)	0
dense_6 (Dense)	(None, 256)	327,936
dropout_3 (Dropout)	(None, 256)	0
dense_7 (Dense)	(None, 1)	257

Total params: 2,586,177 (9.87 MB)
Trainable params: 328,193 (1.25 MB)
Non-trainable params: 2,257,984 (8.61 MB)

B. Comparación general de Enfoques

El Transfer Learning con MobileNetV2 optimiza la extracción, converge rápidamente y generaliza mejor que el CNN Baseline, que corre el riesgo de sobreajuste o subajuste al aprender características desde cero dependiendo del tamaño del conjunto de datos.

C. Interpretación y Análisis de los resultados



Los hallazgos generales del experimento 1:

Como resultado de la gran variabilidad que conlleva usar únicamente 36 imágenes, el experimento se validó sin fuga de datos, lo que evidenció que en situaciones donde los datos son escasos, el desempeño depende del modelo y no de los datos. Esto ubica al aprendizaje por transferencia como la táctica más eficaz en comparación con entrenar una CNN desde el principio. Aunque ambos modelos tienen un balance perfecto y una precisión del 100%, este resultado no valida la hipótesis porque no muestra mejoras en el aprendizaje por transferencia, lo que indica que el problema es de poca dificultad o está mal evaluado debido a un conjunto de datos insuficiente. Por esta razón, se debe examinar la matriz de confusión para comprender los errores dispersos del modelo base en comparación con los concentrados en clases similares del modelo pre entrenado. Se sugiere incrementar la variabilidad del conjunto de datos y utilizar curvas ROC o Grad-CAM para obtener una mayor interpretabilidad.

11. Imágenes usadas en la prueba experimental

Imágenes de entrenamiento por clase.

Se usaron 15 imágenes por cada clase para entrenar el modelo experimental.

- **Visibles**



- **Poco visibles**



- Deterioradas



12. Cronograma Preliminar

Semana	Actividades
Semana 1	Revisión crítica del baseline CNN. Análisis del dataset actual y detección de gaps. Implementación de ResNet50 con fine-tuning.
Semana 2	Implementación de EfficientNet-B0. Primera comparación de métricas entre CNN base y transfer learning.
Semana 3	Etiquetado de imágenes adicionales para YOLO con Roboflow (objetivo: 300 imágenes con bounding boxes). Configuración del entorno YOLOv 8.
Semana 4	Entrenamiento inicial de YOLO v8. Evaluación de primeros resultados de detección. Ajuste de hiperparámetros básico.
Semana 5	Experimento exploratorio con Visión Transformer (ViT-B/16 con fine-tuning). Comparación con resultados de transfer learning clásico.
Semana 6	Implementación del módulo de diagnóstico: reglas de decisión, umbrales de confianza y generación de descripción textual.
Semana 7	Integración de todos los componentes. Pruebas del pipeline completo con imágenes y video de campo. Generación de salidas visuales marcadas.
Semana 8	Redacción del informe final. Preparación de la presentación. Revisión y ajustes finales del sistema.

13. Conclusiones del Avance

A pesar de que el baseline CNN logró una precisión del 44.4% con 240 imágenes, este resultado inicial valida el proyecto al mostrar errores fundamentales como la escasa puntuación F1 en la clase visible_legible, la confusión con señales nocturnas y la ausencia de localización. Estas limitaciones concretas se solucionarán lógicamente a través de transfer learning, reetiquetado y YOLO para corroborar la investigación y desarrollar el sistema definitivo.

Referencias

- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. Proceedings of the 36th International Conference on Machine Learning (ICML).
- Dosovitskiy, A., et al. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. ICLR 2021.
- Jocher, G., et al. (2023). Ultralytics YOLOv8. GitHub. <https://github.com/ultralytics/ultralytics>
- DeVries, T., & Taylor, G. W. (2017). Improved regularization of convolutional neural networks with cutout. arXiv preprint arXiv:1708.04552.
- Cubuk, E. D., Zoph, B., Shlens, J., & Le, Q. V. (2020). Randaugment: Practical automated data augmentation with a reduced search space. CVPR Workshops.
- Muller, S. G., & Hutter, F. (2021). TrivialAugment: Tuning-free yet state-of-the-art data augmentation. ICCV 2021.
- Howard, A. G., et al. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861.
- Notebook del experimento:
<https://drive.google.com/file/d/1aJ6ywQ5KbjRCAsbQ2f1OWSJziUkcVDn6/view?usp=sharing>